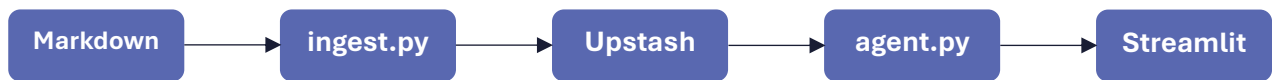


Rapport technique

Le but de ce projet était de réaliser un chatbot de discussion avec notre jumeau virtuel, à intégrer à notre portfolio professionnel, afin de proposer une expérience « originale » aux utilisateurs, qui pourront alors poser des questions au sujet de nos compétences et projet académiques.

La réalisation de ce projet n'est pas un simple appel à une ia générative comme ChatGpt, mais l'utilisation d'une architecture RAG. Le RAG (Retrieval Augmented Generation) est une technique permettant d'améliorer les réponses d'un modèle d'IA générative, en les alimentant de connaissances provenant de bases de données internes. Ici, nous avons donc dû créer nos propres fichiers markdown pour constituer notre propre base de données sur nos informations professionnelles et académiques. Le but de cette technique est de palier le fait que les LLMs ne connaissent pas nos données privées, tout en évitant de les leur fournir.

Pour cela, nous avons suivi un processus de plusieurs étapes vues en cours : **Load** (charger), **split** (découper), **embed** (intégrer), **store** (stocker), **retrieve** (extraire), **generate** (générer une réponse).



Indexation des données

Tout d'abord, j'ai créé un fichier ingest.py afin de permettre l'indexation des données, pour préparer les connaissances de mon chatbot.

Étape 1 : le chargement

Pour cette étape, j'ai utilisé la bibliothèque python Glob afin de lire les fichiers « .md » dans le dossier « data ».

En effet, toutes les données ont été structurées dans des fichiers markdown (pour mes expériences pros, mes compétences, mes projets universitaires, etc), avec des mises en valeurs de titres (avec des « # ») et d'éléments en gras « ** »), pour faciliter la lecture de l'algorithme.

Étape 2 : le découpage

Les documents markdown créés sont un peu longs pour le LLM, c'est donc pour cela qu'il a fallu les découper en chunks (morceaux). J'ai donc choisi de les découper par titre (définis grâce à des « # » en markdown) pour garder une cohérence sémantique par section, donc par exemple, une expérience = un chunk.

Cela a pu se faire grâce à l'expression « contenu.split(« # »)

Étapes 3 et 4 : l'intégration et le stockage

Dans le code j'ai utilisé « `index.upsert(vectors=vect)` » qui sert à transformer le texte en vecteurs numériques (c'est-à-dire une liste de nombres comme [0.3, 0.4, ...] etc) pour être compréhensible par la machine.

Pour stocker les vecteurs créés, on utilise Upstash comme base de données vectorielle.

La création du chatbot

Après l'indexation des données, il faut créer l'agent, dans un fichier python `agent.py` afin de donner vie au chatbot.

Tout d'abord, il faut savoir que lorsque l'on pose une question à l'agent, il ne cherche pas des mots-clés exacts, mais effectue une recherche sémantique, via la similarité des vecteurs (grâce à Upstash) pour trouver les vecteurs les plus proches du sens de la question. Dans le code, j'ai créé une fonction `rechercher_dans_docs` comprenant un `index.query` qui sert donc à aller faire la recherche dans Upstash. Cette expression contient par ailleurs un paramètre `top_k=5`, ce qui signifie que la requête récupère les 5 morceaux de textes les plus pertinents afin de donner assez de contexte à l'ia.

Ensuite, pour l'utilisation du modèle, il a fallu définir les instructions à donner à l'agent pour diriger sa manière de répondre, son comportement. J'ai donc choisi de le faire répondre à la 3^{ème} personne, car je ne souhaite pas qu'il réponde en mon nom directement (au cas où il y aurait des erreurs, ou des bugs de l'ia).

L'interface utilisateur

Afin de permettre l'utilisation de notre agent comme un site internet, il a fallu passer par une application Streamlit, définie dans le fichier `app.py`.

Dans les faits, l'application Streamlit permet de mettre en forme la création de notre agent (par les scripts python) pour en faire une application web interactive.

Ainsi, l'agent n'a plus besoin d'être lancé dans le terminal de notre IDE (VsCode dans mon cas), mais est appelé directement sur mon application Streamlit, avec une interface assez minimaliste et très simple à prendre en main, comme un ChatGpt, masi personnalisé avec les informations de mon portfolio professionnel.

Manuel Technique

Ce manuel technique explique comment lancer le projet dans un IDE, ou en passant directement via le lien de l'application Streamlit déployée.

1. - Prérequis

Il faut avoir Python installé, et utiliser un IDE comme VsCode.

2. Installation de l'environnement

Création de l'environnement virtuel :

```
python -m venv .venv
```

```
.venv\Scripts\activate
```

Puis installation des requirements :

```
pip install -r requirements.txt
```

3. Configuration

Il faut remplir la variable OPENAI_API_KEY grâce à une clé d'API OpenAi.

4. Indexation

Avant de lancer le chatbot il faut alimenter la base de données en remplissant les fichiers Markdown, puis en exécutant le script python ingest.py

5. Lancement

Pour lancer l'application, il faut écrire dans le terminal :

```
Streamlit run app.py
```

L'application s'ouvre alors dans le navigateur dans le localhost.

Pour aller plus loin – version plus complète, faite avec l'IA

Les explications précédentes expliquaient la réalisation du projet la plus simple possible, en comprenant chaque étape et en utilisant l'IA de manière raisonnable, pour correctement comprendre tout le processus.

Pour suivre la tendance de la classe, j'ai également souhaité produire une deuxième version plus élaborée, pour laquelle j'ai clairement plus utilisée l'intelligence artificielle.

Cette version intègre certains atouts :

- Tout d'abord la persistance de la mémoire grâce à Redis. L'application se connecte à une base Upstash Redis pour sauvegarder l'historique des conversations et permettre la gestion de multiples sessions via la sidebar.
- L'ajout d'une nouvelle fonctionnalité de formulaire de contact à l'agent en plus de la recherche documentaire.
- Une interface plus élaborée grâce à l'utilisation poussée des composants natifs Streamlit pour une navigation fluide et ergonomique.